

UNIT-5

INTRODUCTION

A **graph** is a non-linear data structure consisting of nodes and edges. A graph is a finite set of vertices (or nodes) and set of edges which connect a pair of nodes. Or

Graph is a mathematical structure consisting of a set of vertices (also called nodes) $\{v_1, v_2, \dots, v_n\}$ and a set of edges $\{e_1, e_2, \dots, e_n\}$. An edge is a pair of vertices $\{v_i, v_j\}$ $i, j \in \{1 \dots n\}$. The two vertices are called the edge *endpoints*.

Formally: $G = (V, E)$, where V is a set and $E \subseteq V \times V$

A graph may be either *undirected* or *directed*. Intuitively, an undirected edge models a "two-way" or "duplex" connection between its endpoints, while a directed edge is a one-way connection, and is typically drawn as an arrow. A directed edge is often called an *arc*. Mathematically, an undirected edge is an unordered pair of vertices, and an arc is an ordered pair. The maximum number of edges in an undirected graph without a self-loop is $n(n-1)/2$ while a directed graph can have at most n^2 edges.

$G = (V, E)$ undirected if for all $v, w \in V$: $(v, w) \in E \iff (w, v) \in E$. Otherwise directed. For eg.

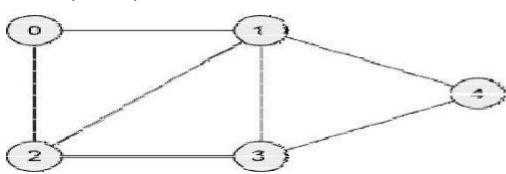


Fig1:Undirect Graph

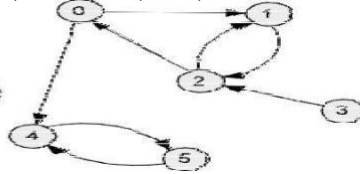


Fig2:Direct Graph

Graphs can be classified by whether or not their edges have **weights**. In **Weighted graph**, edges have a weight. Weight typically shows cost of traversing

Example: weights are distances between cities

In **Unweighted graph**, edges have no weight. Edges simply show connections.

BASIC TERMINOLOGY

- **Edges**, also called arcs, are represented by (u, v) and are either:
- **Directed** if the pairs are ordered (u, v) u the **origin** v the **destination**.
- **Undirected** **End-vertices** of an edge are the **endpoints** of the edge.
- Two vertices are **adjacent** if they are endpoints of the same edge.
- An edge is **incident** on a vertex if the vertex is an endpoint of the edge.
- **Outgoing edges** of a vertex are directed edges that the vertex is the origin.
- **Incoming edges** of a vertex are directed edges that the vertex is the destination.
- **Degree** of a vertex, The degree of a vertex is the number of edges incident to that vertex. In an undirected graph, the number of edges connected to a node is called the degree of that node.
- **Out-degree**, $outdeg(v)$, is the number of outgoing edges.
- **In-degree**, $indeg(v)$, is the number of incoming edges.
- **Parallel edges** or multiple edges are edges of the same type and end-vertices.
- **Self-loop** is an edge with the end vertices the same vertex.
- **Simple graphs** have **no** parallel edges or self-loops.
- **Path** is a sequence of alternating vertices and edges such that each successive vertex is connected by the edge. Frequently only the vertices are listed especially if there are no parallel edges.
- **Cycle** is a path that starts and end at the same vertex.
- **Simple path** is a path with distinct vertices.
- **Directed path** is a path of only directed edges

- **Directed cycle** is a cycle of only directed edges.
- **Sub-graph** is a subset of vertices and edges.
- **Spanning sub-graph** contains all the vertices.
- **Connected graph** has all pairs of vertices connected by at least one path.
- **Connected component** is the maximal connected sub-graph of a disconnected graph.
- **Forest** is a graph without cycles.
- **Tree** is a connected forest (previous type of trees are called rooted trees, these are free trees)
- **Spanning tree** is a spanning sub graph that is also a tree.
- **Simple graph** : A graph or directed graph which does not have any self-loop or parallel edges is called a simple graph.
- **Multi-graph** : A graph which has either a self-loop or parallel edges or both is called a multi-graph.
- **Complete graph** : A graph is complete graph if each vertex is adjacent to every other vertex in graph or there is an edge between any pair of nodes in the graph. An undirected complete graph will contain $n(n-1)/2$ edges.
- **Regular graph** : A graph is regular if every node is adjacent to the same number of nodes. Here every node is adjacent to 3 nodes.
- **Planar graph** : A graph is planar if it can be drawn in a plane without any two intersecting edges.
- **Connected graph** : In a graph G , two vertices v_1 and v_2 are said to be connected if there is path in G from v_1 to v_2 or v_2 to v_1 . Connected graph can be of two types :
 - Strongly connected graph
 - Weakly connected graph
- **Acyclic graph** : If a graph (digraph) does not have any cycle then it is called as acyclic graph.
- **Cyclic graph** : A graph that has cycles is called a cyclic graph.
- **Biconnected graph** : A graph with no articulation points is called a biconnected graph.

Applications of graph :

- Graph is a non-linear data structure and is used to present various operations and algorithms.
- Graphs are used for topological sorting.
- Graphs are used to find shortest paths.
- They are required to minimize some aspect of the graph, such as distance among all the vertices in the graph.

GRAPH REPRESENTATIONS

There are two standard ways of maintaining a graph G in the memory of a computer.

1. The sequential representation
2. The linked representation

1. SEQUENTIAL REPRESENTATION OF GRAPHS

Adjacency Matrix Representation

An **adjacency matrix** is one of the two common ways to represent a graph. The adjacency matrix shows which nodes are **adjacent** to one another. Two nodes are adjacent if there is an edge connecting them. In the case of a directed graph, if node j is adjacent to node i , there is an edge from i to j . In other words, if j is adjacent to i , you can get from i to j by traversing one edge. For a given graph with n nodes, the adjacency matrix will have dimensions of $n \times n$. For an unweighted graph, the adjacency matrix will be populated with Boolean values.

For any given node i , you can determine its adjacent nodes by looking at row $(i, [1...n])$ adjacency matrix. A value of true at (i, j) indicates that there is an edge from node i to node j , and false indicating no edge. In an

undirected graph, the values of (i,j) and (j,i) will be equal. In a weighted graph, the boolean values will be replaced by the weight of the edge connecting the two nodes, with a special value that indicates the absence of an edge.

The memory use of an adjacency matrix is $O(n^2)$.

□ Example undirected graph (assume self-edges not allowed):

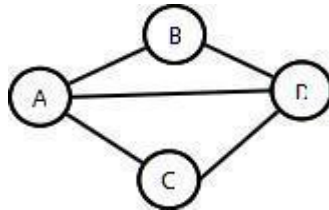
ABCD

A0111

B1001

C1001

D1110



□ Example directed graph (assume self-edges allowed):

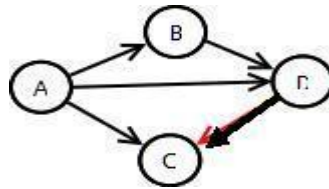
ABCD

A0111

B0001

C0000

D0010



2. LINKED LIST REPRESENTATION OF GRAPH

Adjacency List Representation: The adjacency list is another common representation of a graph. There are many ways to implement this adjacency representation. One way is to have the graph maintain a list of lists, in which the first list is a list of indices corresponding to each node in the graph. Each of these refer to another list that stores a the index of each adjacent node to this one. It might also be useful to associate the weight of each link with the adjacent node in this list.

Example: An undirected graph contains four nodes 1, 2, 3 and 4. 1 is linked to 2 and 3. 2 is linked to 3. 3 is linked to 4.

1-[2,3]

2-[1,3]

3-[1,2,4]

4-[3]

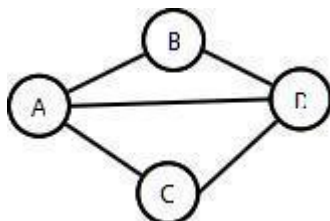
Example (undirected graph):

1. A: B, C, D

2. B: A, D

3. C: A, D

4. D: A, B, C



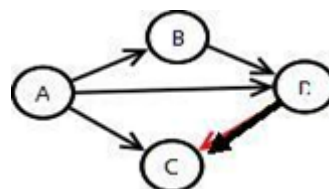
Example (directed graph):

□ 1. A: B, C, D

2. B: D

3. C: Nil

4. D: C



Multi-list for the above UNIGRAPH would

GRAPH TRAVERSAL

Many graph algorithms require one to systematically examine the nodes and edges of a graph G . There are two standard ways that this is done. One way is called a breadth-first search, and the other is called a depth-first search. The breadth-first search will use a queue as an auxiliary structure to hold nodes for future processing, and analogously, the depth-first search will use a stack.

During the execution of our algorithms, each node N of G will be in one of three states, called the status of N , as follows:

STATUS = 1: (Ready state.) The initial State of the node N .

STATUS = 2: (Waiting state.) The node N is on the queue or stack, waiting to be processed,

STATUS = 3: (Processed state.) The node N has been processed.

We now discuss the two searches

1. Breadth-First Search: The general idea behind a breadth-first search beginning at a starting node A is as follows. First we examine the starting node A . Then we examine all the neighbors of A . Then we examine all the neighbors of the neighbors of A . And so on. Naturally, we need to keep track of the neighbors of a node, and we need to guarantee that no node is processed more than once. This is accomplished by using a queue to hold nodes that are waiting to be processed, and by using a field STATUS which tells us the current status of any node. The algorithm follows.

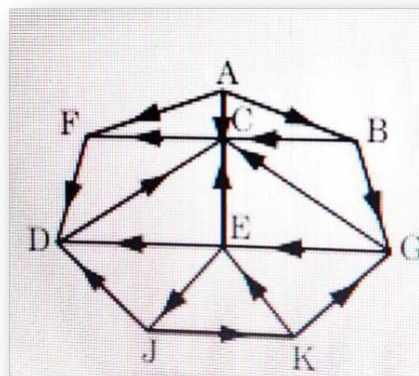
Algorithm

This algorithm executes a breadth first search on a graph G beginning at a starting node A .

1. Initialize all nodes to the ready state (STATUS 1).
2. Put the starting node A in QUEUE and change its status to the waiting state (STATUS = 2).
3. Repeat Steps 4 and 5 until QUEUE is empty:
4. Remove the front node N of QUEUE. Process N and change the status of N to the processed state (STATUS = 3).
5. Add to the rear of QUEUE all the neighbors of N that are in the ready state (STATUS = 1), and change their status to the waiting state (STATUS = 2). [End of Step 3 loop.]
6. Exit.

The above algorithm will process only those nodes which are reachable from the starting node A .

EXAMPLE : Implement BFS algorithm to find the shortest path from node A to J .



Solution: Adjacency list of the graph is :

A :F,C,B

B :G,C

C :F
 D :C
 E :D,C,J
 F :D
 G :C,E
 J :D,K
 K :E,G

- Initially set STATUS=1 for all vertex.
- Now add 'A' to Queue and set STATUS = 2 Queue: A
- Remove A from Queue and set STATUS = 3. And add F, C, B in Queue and change STATUS = 2
- Remove F from Queue, add D in Queue, Now BFS = A, Queue = C, B, D,
- Remove C from Queue, add F, but F is already visited. So no vertex will be added in this step.
BFS = A, F, Queue = B, D
- Remove B from Queue, add G, BFS = A, F, C, B, Queue = D, G
- Remove D from Queue, BFS = A, F, C, B, D, Queue = G
- Remove G from Queue, add E, BFS = A, F, C, B, D, G, Queue = E
- Remove E from Queue, add J, BFS = A, F, C, B, D, G, E, Queue = J
- Remove J, BFS=A,F,C,B,D,G,E,J Queue = Empty

J is our final destination. We now back track from J to find the path from J to A : $J \leftarrow E \leftarrow G \leftarrow B \leftarrow A$

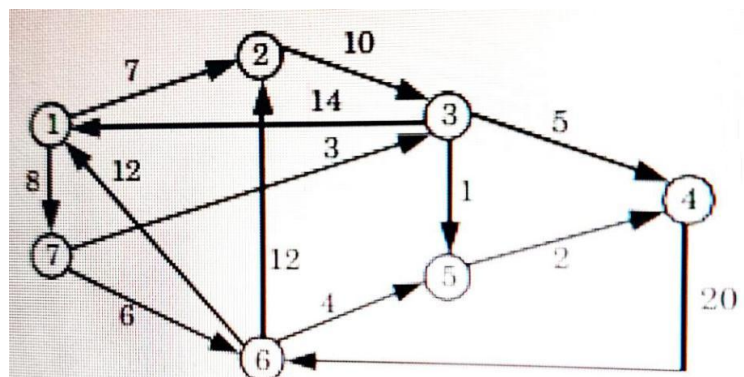
2. Depth-First Search The general idea behind a depth-first Search beginning at a starting node A is as follows. First we examine the starting node A. Then we examine each node N along a path P which begins at A; that is, we process a neighbor of A. then a neighbor of a neighbor of A. and so on. After coming to a "dead end," that is, to the end of the path P, we backtrack on P until we can continue along another path P'. And so on. (This algorithm is similar to the murder traversal of a binary tree, and the algorithm is also similar to the way one might travel through a maze.) P'. And so on (This algorithm is similar to the inorder traversal of a binary tree). The algorithm is very similar to the breadth-first search except now we use a stack instead of the queue. Again, a field STATUS is used to tell us the current status of a node. The algorithm follows.

Algorithm: This algorithm executes a depth-first search on a graph G beginning at its starting node A.

- Initialize all nodes to the ready state (STATUS = 1).
- Push the starting node A onto STACK and change its status to the waiting state (STATUS = 2).
- Repeat Steps 4 and 5 until STACK is empty.
- Pop the top node N of STACK. Process N and change its status to the processed state (STATUS= 3).
- Push onto STACK all the neighbors of N that are still in the ready state (STATUS = 1), and change their status to the waiting state (STATUS = 2).
(End of Step 3 loop.)
- Exit.

Again, the above algorithm will process only those nodes which are reachable from starting node A. Suppose one wants to examine all the nodes in G. Then the algorithm must be modified so that it begins again with another node which we will call B—that is still in the ready state. This node B can be obtained by traversing the list of nodes.

EXAMPLE : Implement DFS algorithm in the graph.



$1 \rightarrow$	2,7
$2 \rightarrow$	3
$3 \rightarrow$	5,4,1
$4 \rightarrow$	6
$5 \rightarrow$	4
$6 \rightarrow$	2,5,1
$7 \rightarrow$	3,6

- | | | |
|--------|--------|---|
| DFS =1 | Stack: | 7 |
| | | 2 |

3. Pop 7 from stack, Push 3, 6; DFS = 1, 7

Stack: 6
3
2

4 . Pop 6 from stack, Push 5; DFS=1,7,6 Stack:

	5
	3

5. Pop 5 from stack, Push 4; DFS=1,7,6,5	Stack:	2
		4
		3
		2

6. Pop 4 from stack; DFS=1,7,6,5,4 Stack: 3
2

7. Pop 3 from stack; DFS=1,7,6,5,4,3 Stack: 2

8. Pop 2 from stack; DFS=1,7,6,5,4,3

Importance of BFS :

- Application of BFS :** Breadth first search can be used to solve many problems in graph theory, for example :

- Importance of DFS :** DFS is very important algorithm as based upon DFS, there are $O(V + E)$ -time algorithms for the following problems :

- Testing whether graph is connected.
- Computing a spanning forest of G .
- Computing the connected components of G .

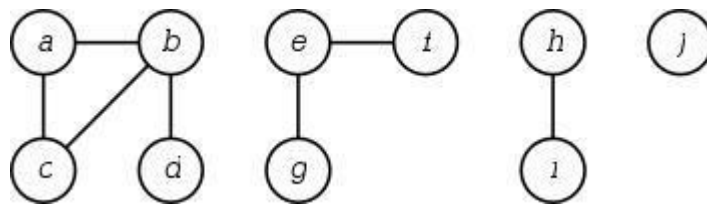
- Computing a path between two vertices of G or reporting that no such path exists.
- Computing a cycle in G or reporting that no such cycle exists

Application of DFS : Algorithms that use depth first search as a building block include :
Finding connected components.

- Topological sorting.
- Finding 2-(edge or vertex)-connected components.
- Finding 3-(edge or vertex)-connected components.
- Finding the bridges of a graph.
- Generating words in order to plot the limit set of a group.
- Finding strongly connected components

CONNECTED COMPONENT

A connected component (or just component) of an undirected graph is a subgraph in which any two vertices are connected to each other by paths, and which is connected to no additional vertices in the super graph.



For example, the graph shown in the illustration above has four connected components $\{a,b,c,d\}$, $\{e,f,g\}$, $\{h,i\}$, and $\{j\}$. A graph that is itself connected has exactly one connected component, consisting of the whole graph.

Strongly connected component : A directed graph is strongly connected if there is a path between all pairs of vertices. A strong component is a maximal subset of strongly connected vertices of subgraph

SPANNING TREE

A spanning tree of an undirected graph is a sub-graph that is a tree which contains all the vertices of graph. A spanning tree of a connected graph G contains all the vertices and has the edges which connect all the vertices. So, the number of edges will be less than the number of nodes.

If graph is not connected, i.e., a graph with n vertices has edges less than $n - 1$ then no spanning tree is possible. A connected graph may have more than one spanning trees.

Minimum spanning tree :

In a weighted graph, a minimum spanning tree is a spanning tree that has minimum weight than all other spanning trees of the same graph.

There are number of techniques for creating a minimum spanning tree for a weighted graph but the most famous methods are Prim's and Kruskal's algorithm.

Kruskal's Algorithm

Kruskal's algorithm is a greedy algorithm in graph theory that finds a minimum spanning tree for a connected weighted graph. This means it finds a subset of the edges that forms a tree that includes every vertex, where the total weight of all the edges in the tree is minimized.

Algorithm

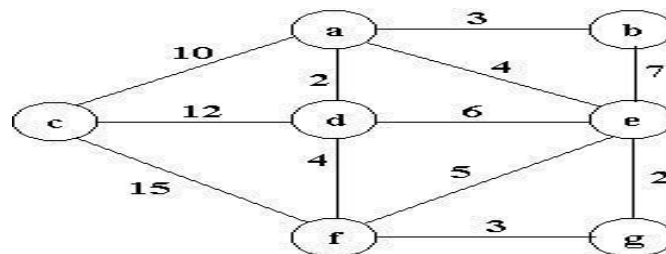
1. create a forest F (a set of trees), where each vertex in the graph is a separate tree
2. create a set S containing all the edges in the graph
3. while S is nonempty and F is not yet spanning
4. remove an edge with minimum weight from S
5. if that edge connects two different trees, then add it to the forest, combining two trees into a single tree

At the termination of the algorithm, the forest forms a minimum spanning forest of the graph. If the graph is connected, the forest has a single component and forms a minimum spanning tree.

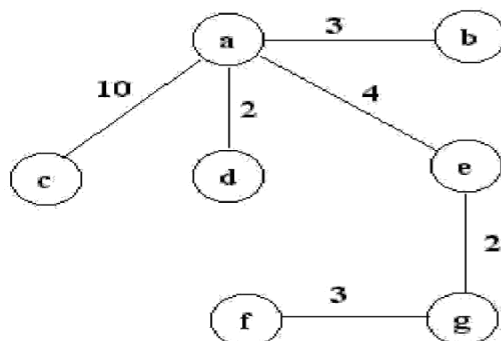
MST-KRUSKAL(G, w)

- 1 $A = \emptyset$
- 2 for each vertex $v \in G, V$
- 3 MAKE-SET (v)
- 4 sort the edges of G, E into nondecreasing order by weight w
- 5 for each edge $(u, v) \in G, E$, taken in nondecreasing order by weight
- 6 if FIND-SET(u) $\neq$ FIND-SET (v)
- 7 $A = A \cup \{(u, v)\}$
- 8 UNION $\{(u, v)\}$
- 9 return A

Example: Trace Kruskal's algorithm in finding a minimum-cost spanning tree for the undirected, weighted graph given below:



edge	ad	eg	ab	fg	ae	df	ef	de	be	ac	cd	cf
weight	2	2	3	3	4	4	5	6	7	10	12	15
insertion status	✓	✓	✓	✓	✓	x	x	x	x	✓	x	x
insertion order	1	2	3	4	5					6		



Therefore The minimum cost is: 24

Prim's Algorithm

The *Prim's* algorithm makes a nature choice of the cut in each iteration – it grows a single tree and adds a light edge in each iteration.

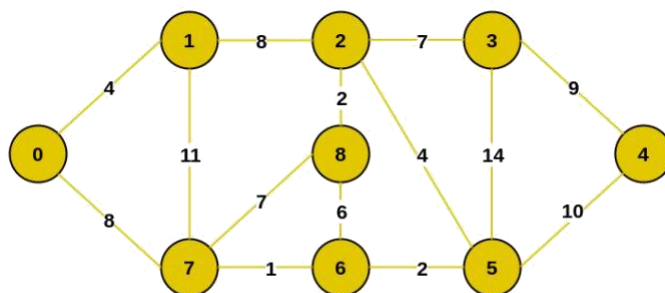
Algorithm

1. Initialize a tree with a single vertex, chosen arbitrarily from the graph.
2. Grow the tree by one edge: of the edges that connect the tree to vertices not yet in the tree, find the minimum-weight edge, and transfer it to the tree.
3. Repeat step 2 (until all vertices are in the tree).

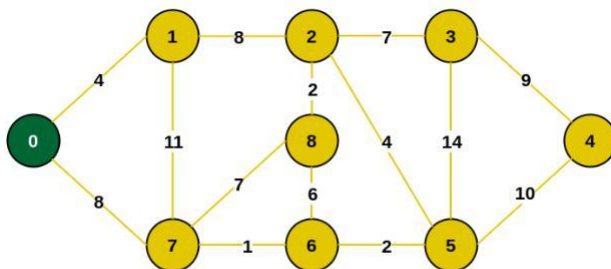
MST-PRIM (G, w, r)

```
1 for each  $u \in G, V$ 
2    $u.key = \infty$ 
3    $u.\pi = NIL$ 
4  $r.key = 0$ 
5  $Q = G, V$ 
6 while  $Q \neq \emptyset$ 
7    $u = \text{EXTRACT-MIN}(Q)$ 
8   for each  $v \in G, \text{Adj}[u]$ 
9     if  $v \in Q$  and  $w(u, v) < v.key$ 
10       $v.\pi = u$ 
11       $v.key = w(u, v)$ 
```

Example: Consider the following graph as an example for which we need to find the Minimum Spanning Tree (MST).



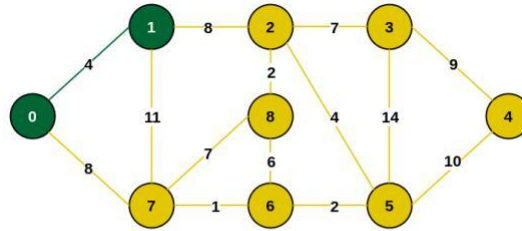
Step 1: Firstly, we select an arbitrary vertex that acts as the starting vertex of the Minimum Spanning Tree. Here we have selected vertex **0** as the starting vertex.



Select an arbitrary starting vertex. Here we have selected 0

0 is selected as starting vertex

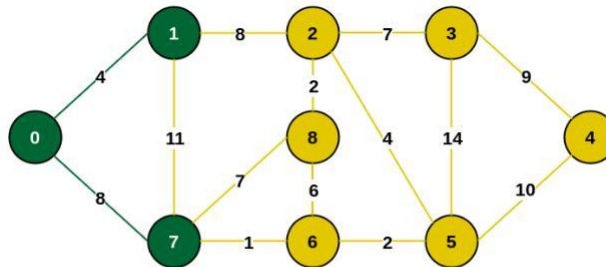
Step 2: All the edges connecting the incomplete MST and other vertices are the edges $\{0, 1\}$ and $\{0, 7\}$. Between these two the edge with minimum weight is $\{0, 1\}$. So include the edge and vertex 1 in the MST.



Minimum weighted edge from MST to other vertices is 0-1 with weight 4

1 is added to the MST

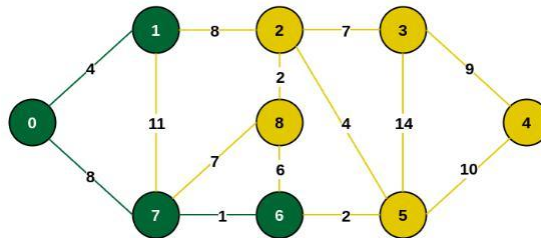
Step 3: The edges connecting the incomplete MST to other vertices are $\{0, 7\}$, $\{1, 7\}$ and $\{1, 2\}$. Among these edges the minimum weight is 8 which is of the edges $\{0, 7\}$ and $\{1, 2\}$. Let us here include the edge $\{0, 7\}$ and the vertex 7 in the MST. [We could have also included edge $\{1, 2\}$ and vertex 2 in the MST].



Minimum weighted edge from MST to other vertices is 0-7 with weight 8

7 is added in the MST

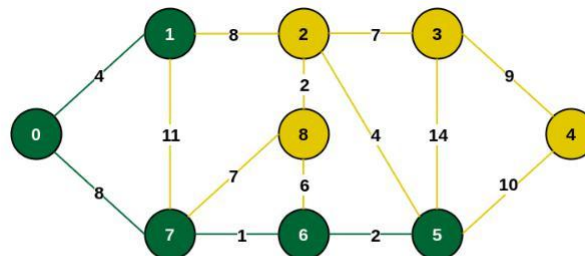
Step 4: The edges that connect the incomplete MST with the fringe vertices are $\{1, 2\}$, $\{7, 6\}$ and $\{7, 8\}$. Add the edge $\{7, 6\}$ and the vertex 6 in the MST as it has the least weight (i.e., 1).



Minimum weighted edge from MST to other vertices is 7-6 with weight 1

6 is added in the MST

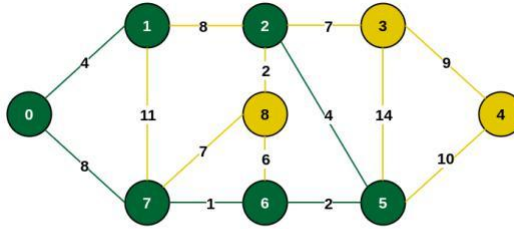
Step 5: The connecting edges now are $\{7, 8\}$, $\{1, 2\}$, $\{6, 8\}$ and $\{6, 5\}$. Include edge $\{6, 5\}$ and vertex 5 in the MST as the edge has the minimum weight (i.e., 2) among them.



Minimum weighted edge from MST to other vertices is 6-5 with weight 2

Include vertex 5 in the MST

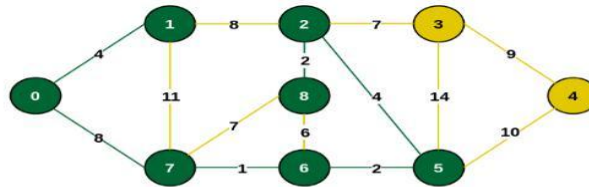
Step 6: Among the current connecting edges, the edge {5, 2} has the minimum weight. So include that edge and the vertex 2 in the MST.



Minimum weighted edge from MST to other vertices is 5-2 with weight 4

Include vertex 2 in the MST

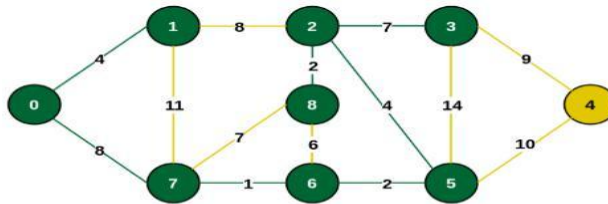
Step 7: The connecting edges between the incomplete MST and the other edges are {2, 8}, {2, 3}, {5, 3} and {5, 4}. The edge with minimum weight is edge {2, 8} which has weight 2. So include this edge and the vertex 8 in the MST.



Minimum weighted edge from MST to other vertices is 2-8 with weight 2

Add vertex 8 in the MST

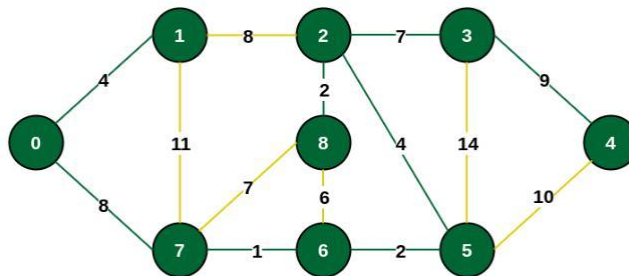
Step 8: See here that the edges {7, 8} and {2, 3} both have same weight which are minimum. But 7 is already part of MST. So we will consider the edge {2, 3} and include that edge and vertex 3 in the MST.



Minimum weighted edge from MST to other vertices is 2-3 with weight 7

Include vertex 3 in MST

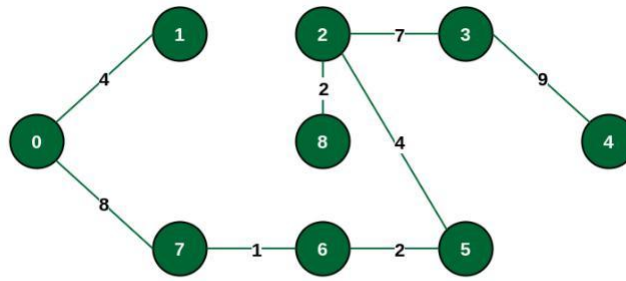
Step 9: Only the vertex 4 remains to be included. The minimum weighted edge from the incomplete MST to 4 is {3, 4}.



Minimum weighted edge from MST to other vertices is 3-4 with weight 9

Include vertex 4 in the MST

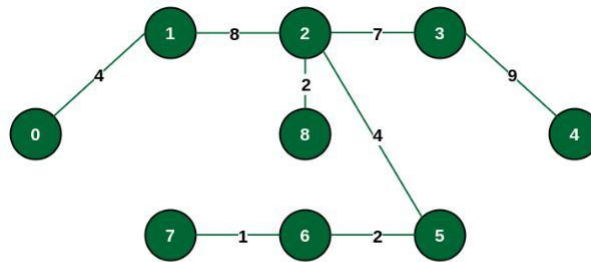
The final structure of the MST is as follows and the weight of the edges of the MST is $(4 + 8 + 1 + 2 + 4 + 2 + 7 + 9) = 37$.



The final structure of MST

The structure of the MST formed using the above method

Note: If we had selected the edge {1, 2} in the third step then the MST would look like the following.



Alternative MST structure

Difference between Kruskal's and Prim's Algorithm

- Prim's algorithm initializes with a node whereas Kruskal algorithm initializes with an edge.
- Kruskal's builds a minimum spanning tree by adding one edge at a time. The next line is always the shortest (minimum weight) ONLY if it does NOT create a cycle.
- Prim's builds a minimum spanning tree by adding one vertex at a time. The next vertex to be added is always the one nearest to a vertex already on the graph.
- In prim's algorithm, graph must be a connected graph while the Kruskal's can function on disconnected graphs too.

Transitive closure of a graph: Given a directed graph, find out if a vertex j is reachable from another vertex i for all vertex pairs (i, j) in the given graph. Here reachable mean that there is a path from vertex i to j . The reach ability matrix is called transitive closure of a graph. The graph is given in the form of adjacency matrix say 'graph[V][V]' where $graph[i][j]$ is 1 if there is an edge from vertex i to vertex j or i is equal to j , otherwise $graph[i][j]$ is 0.

Main idea: a path exists between two vertices i, j , if

There is an edge from i to j ; or

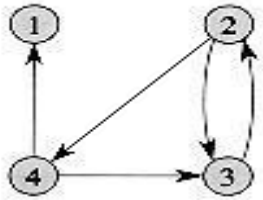
There is a path from i to j going through intermediate vertices which are drawn from set {vertex 1};

Or there is a path from i to j going through intermediate vertices which are drawn from set {vertex 1, 2}; or

There is a path from i to j going through intermediate vertices which are drawn from set {vertex 1, 2, ... $k-1$ }; or

There is a path from i to j going through intermediate vertices which are drawn from set {vertex 1, 2, ... k }; or

There is a path from i to j going through any of the other vertices.



$$T^{(0)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \end{pmatrix} \quad T^{(2)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}$$

$$T^{(3)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \quad T^{(4)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix}$$

Shortest Path algorithm: The shortest path problem is the problem of finding a path between two vertices (or nodes) in a graph such that the sum of the weights of its constituent edges is minimized.

This is analogous to the problem of finding the shortest path between two intersections on a road map: the graph's vertices correspond to intersections and the edges correspond to road segments, each weighted by the length of its road segment. The Minimal Spanning Tree problem is to select a set of edges so that there is a path between each node. The sum of the edge lengths is to be minimized.

The Shortest Path Tree problem is to find the set of edges connecting all nodes such that the sum of the edge lengths from the root to each node is minimized.

Dijkstra Algorithm

Dijkstra's algorithm solves the problem of finding the shortest path from a point in a graph (the *source*) to a destination. It turns out that one can find the shortest paths from a given source to *all* points in a graph in the same time, hence this problem is sometimes called the single-source shortest paths problem.

This problem is related to the spanning tree one. The graph representing all the paths from one vertex to all the others must be a spanning tree - it must include all vertices. There will also be no cycles as a cycle would define more than one path from the selected vertex to at least one other vertex.

DIJKSTRA(G,w,s)

```

1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S =  $\phi$ 
3  Q = G.V
4  while Q  $\neq \phi$ 
5  u = EXTRACT-MIN(Q)
6  S = S  $\cup$  {u}
7  for each vertex v  $\in$  G.Adj[u]
8  RELAX(u,v,w)
```

INITIALIZE-SINGLE-SOURCE(G, s)

```

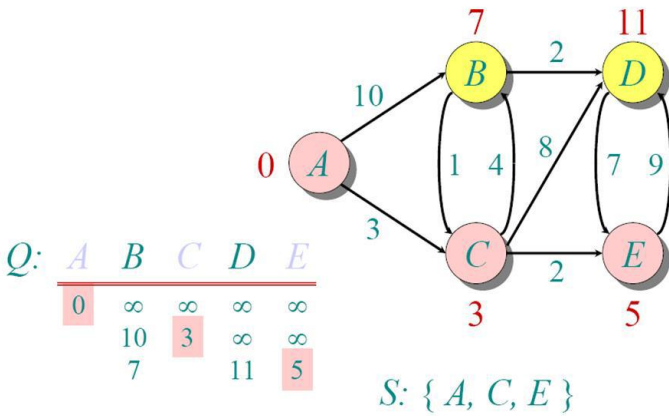
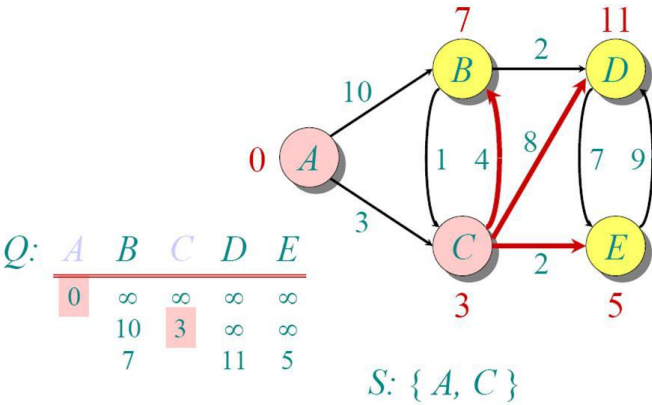
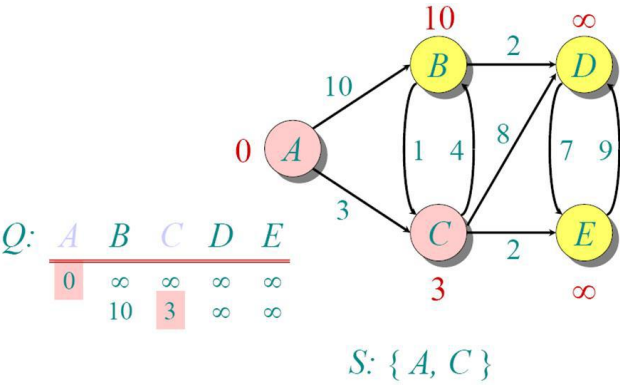
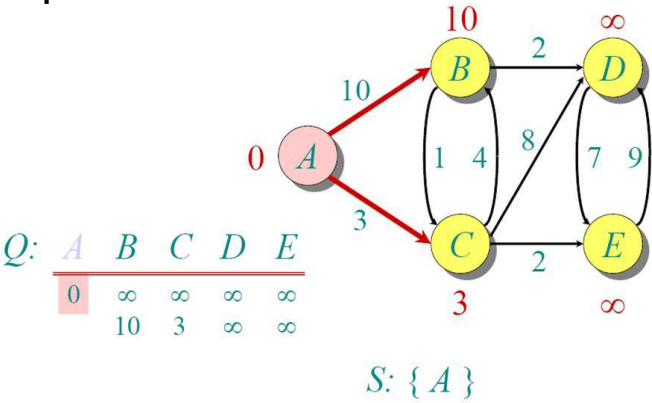
1  for each vertex v  $\in$  G.V
2  v.d =  $\infty$ 
3  v. $\pi$  = NIL
4  s.d = 0
```

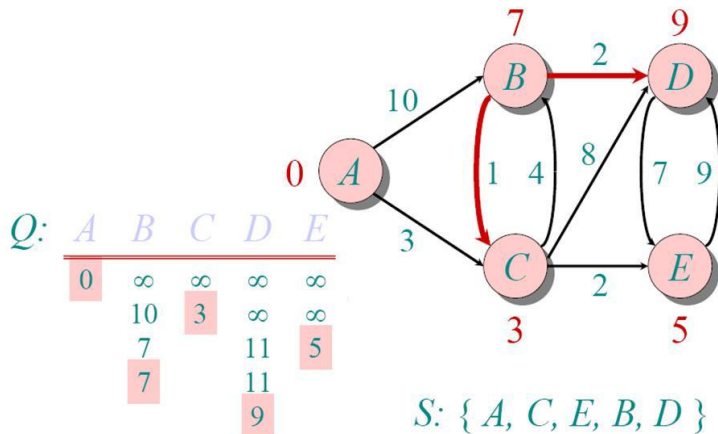
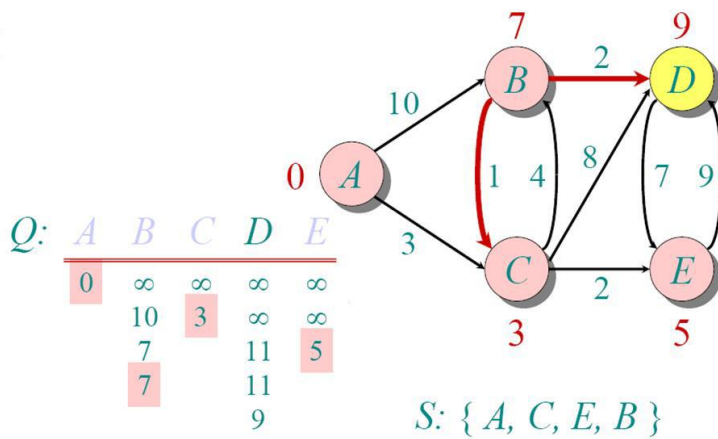
RELAX(u,v,w)

```

1  if v.d > u.d + w(u,v)
2  v.d = u.d + w(u,v)
3  v. $\pi$  = u
```

Example:





Warshall's Algorithm:

Warshall's Algorithm is a graph analysis algorithm for finding shortest paths in a weighted graph with positive or negative edge weights (but with no negative cycles, see below) and also for finding transitive closure of a relation R.

Floyd -Warshall algorithm uses a matrix of lengths D0 as its input. If there is an edge between nodes i and j, then the matrix D0 contains its length at the corresponding coordinates. The diagonal of the matrix contains only zeros. If there is no edge between edges i and j, then the position (i,j) contains positive infinity. In other words, the matrix represents lengths of all paths between nodes that does not contain any intermediate node.

j, then the matrix D0 contains its length at the corresponding coordinates. The diagonal of the matrix contains only zeros. If there is no edge between edges i and j, then the position (i,j) contains positive infinity. In other words, the matrix represents lengths of all paths between nodes that does not contain any intermediate node.

In each iteration of Floyd-Warshall algorithm is this matrix recalculated, so it contains lengths of paths among all pairs of nodes using gradually enlarging set of intermediate nodes. The matrix D1, which is created by the first iteration of the procedure, contains paths among all nodes using exactly one (predefined) intermediate node. D2 contains lengths using two predefined intermediate nodes. Finally the matrix Dn uses n intermediate nodes. This transformation can be described using the following formula:

$$d_{ij}^k = \begin{cases} w_{ij} & \text{if } k=0 \\ \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)}) & \text{if } k \geq 1 \end{cases}$$

Let d_{ij}^k be the weight of a shortest path from vertex i to vertex j for which all intermediate vertices are in the set $\{1, 2; \dots; k\}$. When $k = 0$, a path from vertex i to vertex j with no intermediate vertex numbered higher than 0 has no intermediate vertices at all. Such a path has at most one edge, and hence $d_{ij}^0 = w_{ij}$.

We can give a recursive formulation of π_k ij . When $k = 0$, a shortest path from i to j has no intermediate vertices at all. Thus,

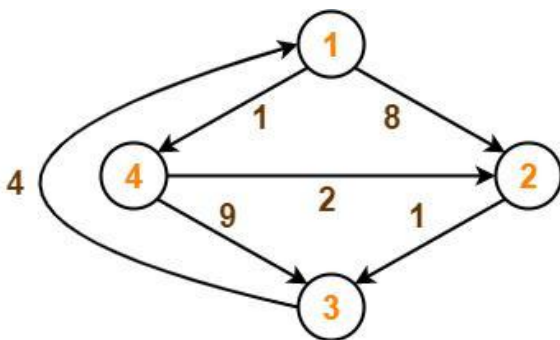
$$\pi_{ij}^0 = \begin{cases} \text{NIL} & \text{if } i = j \text{ or } w_{ij} = \infty \\ i & \text{if } i \neq j \text{ and } w_{ij} < \infty \end{cases}$$

$$\pi_{ij}^k = \begin{cases} \pi_{ij}^{(k-1)} & \text{if } d_{ij}^{(k-1)} \leq d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \\ \pi_{kj}^{(k-1)} & \text{if } d_{ij}^{(k-1)} > d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \end{cases}$$

FLOYD-WARSHALL (W)

1. $n = \text{rows}(W)$
2. $D^0 = W$
3. for $k = 1$ to n
4. do for $i = 1$ to n
5. Do for $j = 1$ to n
6. Do $d_{ij}^0 = \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$
7. return D^n

Example: Consider the following directed weighted graph-



Using Floyd Warshall Algorithm, find the shortest path distance between every pair of vertices.

Solution-

Step-01:

- Write the initial distance matrix.
- It represents the distance between every pair of vertices in the form of given weights.
- For diagonal elements (representing self-loops), distance value = 0.
- For vertices having a direct edge between them, distance value = weight of that edge.
- For vertices having no direct edge between them, distance value = ∞ .

Initial distance matrix for the given graph is-

$$D_0 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & \infty & 0 & \infty \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

Step-02:

Using Floyd Warshall Algorithm, write the following 4 matrices-

$$D_1 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & \infty & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 9 & 0 \end{bmatrix} \end{matrix}$$

$$D_2 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ \infty & 0 & 1 & \infty \\ 4 & 12 & 0 & 5 \\ \infty & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$D_3 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 8 & 9 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 12 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

$$D_4 = \begin{matrix} & \begin{matrix} 1 & 2 & 3 & 4 \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & 3 & 4 & 1 \\ 5 & 0 & 1 & 6 \\ 4 & 7 & 0 & 5 \\ 7 & 2 & 3 & 0 \end{bmatrix} \end{matrix}$$

*****End of Unit-5*****